

RentalSystem

Sistema de Aluguel de Veículos — API REST

Etapa 3: Testes e Documentação

Aluno: Kaike Yoshioka

Data dos testes: 03/10/2026

Tecnologias: C# / ASP.NET Core (.NET 10), Entity Framework Core, SQL Server Express, Swagger (Swashbuckle)

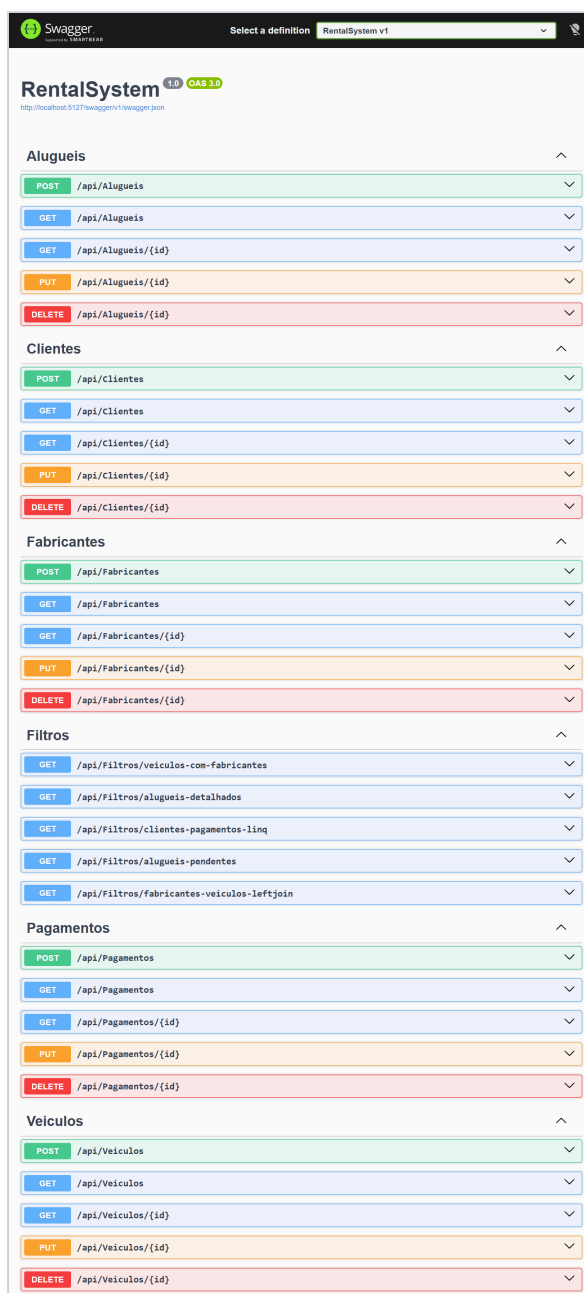
Resultado: 50 de 50 testes aprovados

1. Integração do Swagger

O Swagger foi integrado ao projeto pelo pacote `Swashbuckle.AspNetCore`, configurado no `Program.cs`:

```
builder.Services.AddEndpointsApiExplorer();  
builder.Services.AddSwaggerGen();  
app.UseSwagger();  
app.UseSwaggerUI();
```

Com a API em execução (`dotnet run --project RentalSystem --launch-profile http`), a interface fica disponível em `http://localhost:5127/swagger` e lista todos os endpoints dos seis controllers. Todos os testes deste relatório foram executados por essa interface (botões *Try it out* e *Execute*).



2. Documentação das APIs

URL base: `http://localhost:5127/api`. Requisições e respostas em JSON; datas no formato `2026-10-01T10:00:00`.

Código	Significado
200 OK	Operação realizada. No POST retorna o registro criado com o <code>id</code> gerado.
400 Bad Request	Dados inválidos, Id da rota diferente do Id do corpo, ou Id relacionado (fabricante, cliente, veículo, aluguel) inexistente.
404 Not Found	Registro com o Id informado não existe.
409 Conflict	CPF já cadastrado.
500 Internal Server Error	Erro inesperado no servidor ou no banco.

Fabricantes — `/api/Fabricantes`

Corpo (JSON): `{ "id": 0, "nome": "Honda" }`

Método	Rota	Descrição	Parâmetros	Respostas
POST	<code>/api/Fabricantes</code>	Cadastra	Corpo JSON	200, 400, 500
GET	<code>/api/Fabricantes</code>	Lista todos	—	200, 500
GET	<code>/api/Fabricantes/{id}</code>	Busca por Id	id (rota)	200, 404, 500
PUT	<code>/api/Fabricantes/{id}</code>	Atualiza	id (rota) + corpo JSON com o mesmo id	200, 400, 404, 500
DELETE	<code>/api/Fabricantes/{id}</code>	Remove	id (rota)	200, 404, 500

Veículos — `/api/Veiculos`

Corpo (JSON): `{ "id": 0, "modelo": "Civic", "anoFabricacao": 2022, "quilometragem": 30000, "fabricanteId": 1 }`

Método	Rota	Descrição	Parâmetros	Respostas
POST	<code>/api/Veiculos</code>	Cadastra	Corpo JSON	200, 400, 500
GET	<code>/api/Veiculos</code>	Lista todos	—	200, 500
GET	<code>/api/Veiculos/{id}</code>	Busca por Id	id (rota)	200, 404, 500
PUT	<code>/api/Veiculos/{id}</code>	Atualiza	id (rota) + corpo JSON com o mesmo id	200, 400, 404, 500
DELETE	<code>/api/Veiculos/{id}</code>	Remove	id (rota)	200, 404, 500

O `fabricanteId` precisa existir (senão retorna 400).

Clientes — `/api/Clientes`

Corpo (JSON): { "id": 0, "nome": "Carlos Almeida", "cpf": "111.222.333-44", "email": "carlos@email.com" }

Método	Rota	Descrição	Parâmetros	Respostas
POST	/api/Clientes	Cadastra	Corpo JSON	200, 400, 409, 500
GET	/api/Clientes	Lista todos	—	200, 500
GET	/api/Clientes/{id}	Busca por Id	id (rota)	200, 404, 500
PUT	/api/Clientes/{id}	Atualiza	id (rota) + corpo JSON com o mesmo id	200, 400, 404, 409, 500
DELETE	/api/Clientes/{id}	Remove	id (rota)	200, 404, 500

Nome, CPF e e-mail são obrigatórios; o e-mail precisa ser válido e o CPF é único.

Aluguéis — /api/Alugueis

Corpo (JSON): { "id": 0, "dataInicio": "...", "dataFimPrevista": "...", "dataDevolucao": null, "quilometragemInicial": 30500, "quilometragemFinal": null, "valorDiaria": 120.00, "valorTotal": null, "clienteId": 1, "veiculoId": 1 }

Método	Rota	Descrição	Parâmetros	Respostas
POST	/api/Alugueis	Cadastra	Corpo JSON	200, 400, 500
GET	/api/Alugueis	Lista todos	—	200, 500
GET	/api/Alugueis/{id}	Busca por Id	id (rota)	200, 404, 500
PUT	/api/Alugueis/{id}	Atualiza	id (rota) + corpo JSON com o mesmo id	200, 400, 404, 500
DELETE	/api/Alugueis/{id}	Remove	id (rota)	200, 404, 500

clienteId e veiculoId precisam existir. A devolução é registrada pelo PUT, informando dataDevolucao, quilometragemFinal e valorTotal.

Pagamentos — /api/Pagamentos

Corpo (JSON): { "id": 0, "metodoPagamento": "PIX", "valorPago": 360.00, "dataPagamento": "...", "aluguelId": 1 }

Método	Rota	Descrição	Parâmetros	Respostas
POST	/api/Pagamentos	Cadastra	Corpo JSON	200, 400, 500
GET	/api/Pagamentos	Lista todos	—	200, 500
GET	/api/Pagamentos/{id}	Busca por Id	id (rota)	200, 404, 500
PUT	/api/Pagamentos/{id}	Atualiza	id (rota) + corpo JSON com o mesmo id	200, 400, 404, 500
DELETE	/api/Pagamentos/{id}	Remove	id (rota)	200, 404, 500

O aluguelId precisa existir.

Filtros — /api/Filtros

Todos são GET, sem parâmetros, e retornam 200 (lista) ou 500.

Rota	Join	Tabelas	Retorno
/veiculos-com-fabricantes	INNER JOIN	Veiculos × Fabricantes	id, modelo, anoFabricacao, fabricante
/alugueis-detalhados	INNER JOIN	Alugueis × Clientes × Veiculos	idAluguel, dataInicio, clienteNome, veiculoModelo, valorTotal
/clientes-pagamentos-linq	INNER JOIN (LINQ)	Clientes × Alugueis × Pagamentos	nomeCliente, cpf, dataAluguel, valorPago, metodo
/alugueis-pendentes	INNER JOIN + WHERE	Alugueis × Clientes × Veiculos (sem devolução)	id, cliente, veiculo, dataFimPrevista
/fabricantes-veiculos-leftjoin	LEFT JOIN	Fabricantes × Veiculos	fabricante, veiculo ("Sem veículos" quando não há)

3. Relatório de Testes

Os testes seguiram um fluxo lógico, partindo de um banco de dados vazio: cadastro de fabricantes → veículos → clientes → aluguéis → pagamentos, depois as consultas com joins e, por fim, as exclusões. Para cada endpoint foram testados o caso de sucesso e os principais casos de erro. Nas páginas seguintes, cada teste traz a chamada, a entrada, o resultado esperado, o resultado obtido e o print da tela do Swagger.

#	Teste	Chamada	Esperado	Obtido	
1	Cadastrar fabricante Honda	POST /api/Fabricantes	200	200	Aprovado
2	Cadastrar fabricante Toyota	POST /api/Fabricantes	200	200	Aprovado
3	Cadastrar fabricante Fiat (ficará sem veículos)	POST /api/Fabricantes	200	200	Aprovado
4	Cadastrar fabricante com nome vazio (erro de validação)	POST /api/Fabricantes	400	400	Aprovado
5	Listar todos os fabricantes	GET /api/Fabricantes	200	200	Aprovado
6	Buscar fabricante por Id	GET /api/Fabricantes/1	200	200	Aprovado
7	Atualizar nome do fabricante Fiat	PUT /api/Fabricantes/3	200	200	Aprovado
8	Atualizar fabricante inexistente	PUT /api/Fabricantes/999	404	404	Aprovado
9	Atualizar com Id da rota diferente do corpo	PUT /api/Fabricantes/1	400	400	Aprovado
10	Cadastrar veículo Civic (Honda)	POST /api/Veiculos	200	200	Aprovado
11	Cadastrar veículo Corolla (Toyota)	POST /api/Veiculos	200	200	Aprovado
12	Cadastrar veículo com fabricante inexistente	POST /api/Veiculos	400	400	Aprovado
13	Listar todos os veículos	GET /api/Veiculos	200	200	Aprovado
14	Buscar veículo por Id	GET /api/Veiculos/1	200	200	Aprovado
15	Buscar veículo inexistente	GET /api/Veiculos/999	404	404	Aprovado
16	Atualizar quilometragem do Civic	PUT /api/Veiculos/1	200	200	Aprovado
17	Cadastrar cliente Carlos Almeida	POST /api/Clientes	200	200	Aprovado
18	Cadastrar cliente Ana Beatriz	POST /api/Clientes	200	200	Aprovado
19	Cadastrar cliente com CPF já existente	POST /api/Clientes	409	409	Aprovado
20	Cadastrar cliente com e-mail inválido	POST /api/Clientes	400	400	Aprovado
21	Listar todos os clientes	GET /api/Clientes	200	200	Aprovado
22	Buscar cliente por Id	GET /api/Clientes/1	200	200	Aprovado
23	Atualizar e-mail da cliente Ana	PUT /api/Clientes/2	200	200	Aprovado
24	Registrar aluguel: Carlos aluga o Civic	POST /api/Alugueis	200	200	Aprovado

25	Registrar aluguel: Ana aluga o Corolla	POST /api/Alugueis	200	200	Aprovado
26	Registrar aluguel com cliente inexistente	POST /api/Alugueis	400	400	Aprovado
27	Listar todos os aluguéis	GET /api/Alugueis	200	200	Aprovado
28	Buscar aluguel por Id	GET /api/Alugueis/1	200	200	Aprovado
29	Registrar a devolução do Civic (data, km final e valor total)	PUT /api/Alugueis/1	200	200	Aprovado
30	Registrar pagamento do aluguel do Carlos (PIX)	POST /api/Pagamentos	200	200	Aprovado
31	Registrar pagamento de aluguel inexistente	POST /api/Pagamentos	400	400	Aprovado
32	Listar todos os pagamentos	GET /api/Pagamentos	200	200	Aprovado
33	Buscar pagamento por Id	GET /api/Pagamentos/1	200	200	Aprovado
34	Atualizar método de pagamento para Cartão de Crédito	PUT /api/Pagamentos/1	200	200	Aprovado
35	Veículos com fabricantes (INNER JOIN)	GET /api/Filtros/veiculos-com-fabricantes	200	200	Aprovado
36	Aluguéis detalhados com cliente e veículo (INNER JOIN)	GET /api/Filtros/alugueis-detalhados	200	200	Aprovado
37	Pagamentos por cliente (INNER JOIN, sintaxe LINQ)	GET /api/Filtros/clientes-pagamentos-linq	200	200	Aprovado
38	Aluguéis pendentes de devolução (INNER JOIN + WHERE)	GET /api/Filtros/alugueis-pendentes	200	200	Aprovado
39	Fabricantes e veículos, incluindo sem veículos (LEFT JOIN)	GET /api/Filtros/fabricantes-veiculos-leftjoin	200	200	Aprovado
40	Cadastrar fabricante temporário	POST /api/Fabricantes	200	200	Aprovado
41	Cadastrar veículo temporário	POST /api/Veiculos	200	200	Aprovado
42	Cadastrar cliente temporário	POST /api/Clientes	200	200	Aprovado
43	Registrar aluguel temporário	POST /api/Alugueis	200	200	Aprovado
44	Registrar pagamento temporário	POST /api/Pagamentos	200	200	Aprovado
45	Excluir pagamento	DELETE /api/Pagamentos/2	200	200	Aprovado
46	Excluir aluguel	DELETE /api/Alugueis/3	200	200	Aprovado
47	Excluir veículo	DELETE /api/Veiculos/3	200	200	Aprovado
48	Excluir cliente	DELETE /api/Clientes/3	200	200	Aprovado
49	Excluir fabricante	DELETE /api/Fabricantes/4	200	200	Aprovado
50	Excluir novamente o mesmo fabricante (já removido)	DELETE /api/Fabricantes/4	404	404	Aprovado

Teste 1: Cadastrar fabricante Honda

Chamada	POST /api/Fabricantes
Entrada	<code>{"nome": "Honda"}</code>
Esperado	HTTP 200
Obtido	HTTP 200 — <code>{"id":1,"nome":"Honda"}</code>
Resultado	Aprovado

POST /api/Fabricantes

Parameters

No parameters

Request body

application/json

Edit Value | Schema

```
{  
  "nome": "Honda"  
}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Fabricantes

Server response

Code	Details
200	Response body <pre>{ "id": 1, "nome": "Honda" }</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Sat, 03 Oct 2026 21:42:56 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Teste 2: Cadastrar fabricante Toyota

Chamada	POST /api/Fabricantes
Entrada	<code>{"nome": "Toyota"}</code>
Esperado	HTTP 200
Obtido	HTTP 200 — <code>{"id": 2, "nome": "Toyota"}</code>
Resultado	Aprovado

POST /api/Fabricantes

Parameters

No parameters

Request body

application/json

Edit Value | Schema

```
{  
  "nome": "Toyota"  
}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Fabricantes

Server response

Code	Details
200	Response body <pre>{ "id": 2, "nome": "Toyota" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Sat, 03 Oct 2026 21:42:58 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Teste 3: Cadastrar fabricante Fiat (ficará sem veículos)

Chamada	POST /api/Fabricantes
Entrada	<code>{"nome": "Fiat"}</code>
Esperado	HTTP 200
Obtido	HTTP 200 — <code>{"id": 3, "nome": "Fiat"}</code>
Resultado	Aprovado

POST /api/Fabricantes

Parameters

Cancel

Reset

No parameters

Request body

application/json

Edit Value | Schema

```
{
  "nome": "Fiat"
}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Fabricantes

Server response

Code	Details
200	Response body <pre>{ "id": 3, "nome": "Fiat" }</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Sat, 03 Oct 2026 21:42:59 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Teste 4: Cadastrar fabricante com nome vazio (erro de validação)

Chamada	POST /api/Fabricantes
Entrada	<code>{"nome": ""}</code>
Esperado	HTTP 400
Obtido	HTTP 400 — <code>{"type": "https://tools.ietf.org/html/rfc9110#section-15.5.1", "title": "One or more validation errors occurred.", "status": 400, "errors": {"Nome": ["O nome do fabricante é obrigatório."]}, "traceId": "00-6aaea1893e44b6c17165bbcccdedf515-eefd606e6e839d6b-00"}</code>
Resultado	Aprovado

POST /api/Fabricantes

Parameters Cancel Reset

No parameters

Request body application/json

Edit Value | Schema

```
{  "nome": ""}
```

Execute Clear

Responses

Request URL

http://localhost:5127/api/Fabricantes

Server response

Code	Details
400	Error: Bad Request

Undocumented

Response body

```
{  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",  "title": "One or more validation errors occurred.",  "status": 400,  "errors": {    "Nome": [      "O nome do fabricante é obrigatório."    ]  },  "traceId": "00-6aaea1893e44b6c17165bbcccdedf515-eefd606e6e839d6b-00"}
```

Download

Response headers

```
content-type: application/problem+json; charset=utf-8
date: Sat, 03 Oct 2026 21:43:00 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Teste 5: Listar todos os fabricantes

Chamada	GET /api/Fabricantes
Entrada	—
Esperado	HTTP 200
Obtido	HTTP 200 — [{"id":1,"nome":"Honda"}, {"id":2,"nome":"Toyota"}, {"id":3,"nome":"Fiat"}]
Resultado	Aprovado

GET /api/Fabricantes

Parameters

No parameters

ExecuteClear

Responses

Request URL

http://localhost:5127/api/Fabricantes

Server response

Code

Details

200

Response body

```
[
  {
    "id": 1,
    "nome": "Honda"
  },
  {
    "id": 2,
    "nome": "Toyota"
  },
  {
    "id": 3,
    "nome": "Fiat"
  }
]
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Sat, 03 Oct 2026 21:43:01 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Teste 6: Buscar fabricante por Id

Chamada	GET /api/Fabricantes/{id} id = 1
Entrada	—
Esperado	HTTP 200
Obtido	HTTP 200 — {"id":1,"nome":"Honda"}
Resultado	Aprovado

GET /api/Fabricantes/{id}

Parameters

Name

Description

id * required

integer(\$int32)

(path)

1

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Fabricantes/1

Server response

Code

Details

200

Response body

```
{
  "id": 1,
  "nome": "Honda"
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Sat, 03 Oct 2026 21:43:02 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Teste 7: Atualizar nome do fabricante Fiat

Chamada	PUT /api/Fabricantes/{id} id = 3
Entrada	<code>{"id":3,"nome":"Fiat Automóveis"}</code>
Esperado	HTTP 200
Obtido	HTTP 200 — <code>{"id":3,"nome":"Fiat Automóveis"}</code>
Resultado	Aprovado

PUT /api/Fabricantes/{id}

Parameters

Cancel

Reset

Name	Description
id * required integer(\$int32) (path)	3

Request body

application/json

Edit Value | Schema

```
{  "id": 3,  "nome": "Fiat Automóveis"}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Fabricantes/3

Server response

Code	Details
200	Response body <pre>{ "id": 3, "nome": "Fiat Automóveis"}</pre>  Download Response headers content-type: application/json; charset=utf-8 date: Sat, 03 Oct 2026 21:43:02 GMT server: Kestrel transfer-encoding: chunked

Responses

Teste 8: Atualizar fabricante inexistente

Chamada	PUT /api/Fabricantes/{id} id = 999
Entrada	<code>{"id":999,"nome":"Inexistente"}</code>
Esperado	HTTP 404
Obtido	HTTP 404 — Fabricante não encontrado.
Resultado	Aprovado

PUT /api/Fabricantes/{id}

Parameters

Cancel

Reset

Name	Description
id * required	
integer(\$int32)	999
(path)	

Request body

application/json

Edit Value | Schema

```
{  "id": 999,  "nome": "Inexistente"}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Fabricantes/999

Server response

Code	Details
404	Error: Not Found

Response body

Fabricante não encontrado.

Download

Response headers

content-type: text/plain; charset=utf-8
date: Sat, 03 Oct 2026 21:43:04 GMT
server: Kestrel
transfer-encoding: chunked

Responses

Teste 9: Atualizar com Id da rota diferente do corpo

Chamada	PUT /api/Fabricantes/{id} id = 1
Entrada	<code>{"id":2,"nome":"Honda"}</code>
Esperado	HTTP 400
Obtido	HTTP 400 — IDs divergentes.
Resultado	Aprovado

PUT /api/Fabricantes/{id}

Parameters

Cancel

Reset

Name	Description
id * required integer(\$int32) (path)	1

Request body

application/json

Edit Value | Schema

```
{  "id": 2,  "nome": "Honda"}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Fabricantes/1

Server response

Code	Details
400 <i>Undocumented</i>	Error: Bad Request

Response body

IDs divergentes.

Download

Response headers

content-type: text/plain; charset=utf-8
date: Sat, 03 Oct 2026 21:43:05 GMT
server: Kestrel
transfer-encoding: chunked

Responses

Teste 10: Cadastrar veículo Civic (Honda)

Chamada	POST /api/Veiculos
Entrada	{ "modelo": "Civic", "anoFabricacao": 2022, "quilometragem": 30000, "fabricanteId": 1 }
Esperado	HTTP 200
Obtido	HTTP 200 — { "id": 1, "modelo": "Civic", "anoFabricacao": 2022, "quilometragem": 30000, "fabricanteId": 1 }
Resultado	Aprovado

POST /api/Veiculos

Parameters

No parameters

Request body

application/json

Edit Value | Schema

```
{
  "modelo": "Civic",
  "anoFabricacao": 2022,
  "quilometragem": 30000,
  "fabricanteId": 1
}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Veiculos

Server response

Code

Details

200

Response body

```
{
  "id": 1,
  "modelo": "Civic",
  "anoFabricacao": 2022,
  "quilometragem": 30000,
  "fabricanteId": 1
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Sat, 03 Oct 2026 21:43:06 GMT
server: Kestrel
transfer-encoding: chunked
```

Teste 11: Cadastrar veículo Corolla (Toyota)

Chamada	POST /api/Veiculos
Entrada	{ "modelo": "Corolla", "anoFabricacao": 2023, "quilometragem": 12000, "fabricanteId": 2 }
Esperado	HTTP 200
Obtido	HTTP 200 — { "id": 2, "modelo": "Corolla", "anoFabricacao": 2023, "quilometragem": 12000, "fabricanteId": 2 }
Resultado	Aprovado

POST /api/Veiculos

Parameters

No parameters

Request body

application/json

Edit Value | Schema

```
{
  "modelo": "Corolla",
  "anoFabricacao": 2023,
  "quilometragem": 12000,
  "fabricanteId": 2
}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Veiculos

Server response

Code

Details

200

Response body

```
{
  "id": 2,
  "modelo": "Corolla",
  "anoFabricacao": 2023,
  "quilometragem": 12000,
  "fabricanteId": 2
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Sat, 03 Oct 2026 21:43:06 GMT
server: Kestrel
transfer-encoding: chunked
```

Teste 12: Cadastrar veículo com fabricante inexistente

Chamada	POST /api/Veiculos
Entrada	<pre>{ "modelo": "Onix", "anoFabricacao": 2021, "quilometragem": 5000, "fabricanteId": 999 }</pre>
Esperado	HTTP 400
Obtido	HTTP 400 — Fabricante não existe.
Resultado	Aprovado

POST /api/Veiculos

Parameters

Cancel

Reset

No parameters

Request body

application/json

Edit Value | Schema

```
{  "modelo": "Onix",  "anoFabricacao": 2021,  "quilometragem": 5000,  "fabricanteId": 999}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Veiculos

Server response

Code

Details

400

Error: Bad Request

Undocumented

Response body

Fabricante não existe.

Download

Response headers

content-type: text/plain; charset=utf-8
date: Sat, 03 Oct 2026 21:43:07 GMT
server: Kestrel
transfer-encoding: chunked

Responses

Teste 13: Listar todos os veículos

Chamada	GET /api/Veiculos
Entrada	—
Esperado	HTTP 200
Obtido	HTTP 200 — [{"id":1,"modelo":"Civic","anoFabricacao":2022,"quilometragem":30000,"fabricanteId":1}, { "id":2,"modelo":"Corolla","anoFabricacao":2023,"quilometragem":12000,"fabricanteId":2}]
Resultado	Aprovado

GET /api/Veiculos

Parameters

Cancel

No parameters

ExecuteClear

Responses

Request URL

http://localhost:5127/api/Veiculos

Server response

Code	Details
200	Response body <pre>[{ "id": 1, "modelo": "Civic", "anoFabricacao": 2022, "quilometragem": 30000, "fabricanteId": 1 }, { "id": 2, "modelo": "Corolla", "anoFabricacao": 2023, "quilometragem": 12000, "fabricanteId": 2 }]</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Sat, 03 Oct 2026 21:43:07 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Teste 14: Buscar veículo por Id

Chamada	GET /api/Veiculos/{id} id = 1
Entrada	—
Esperado	HTTP 200
Obtido	HTTP 200 — { "id":1, "modelo": "Civic", "anoFabricacao":2022, "quilometragem":30000, "fabricanteId":1 }
Resultado	Aprovado

GET /api/Veiculos/{id}

Parameters

Cancel

Name	Description
id * required	
integer(\$int32)	1
(path)	

ExecuteClear

Responses

Request URL

http://localhost:5127/api/Veiculos/1

Server response

Code	Details
200	Response body { "id": 1, "modelo": "Civic", "anoFabricacao": 2022, "quilometragem": 30000, "fabricanteId": 1 } Download Response headers content-type: application/json; charset=utf-8 date: Sat, 03 Oct 2026 21:43:08 GMT server: Kestrel transfer-encoding: chunked

Responses

Teste 15: Buscar veículo inexistente

Chamada	GET /api/Veiculos/{id} id = 999
Entrada	—
Esperado	HTTP 404
Obtido	HTTP 404 — Veículo não encontrado.
Resultado	Aprovado

GET /api/Veiculos/{id}

Parameters

Cancel

Name	Description
id * required	
integer(\$int32)	999
(path)	

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Veiculos/999

Server response

Code	Details
404	Error: Not Found

Undocumented

Response body

Veículo não encontrado.

Download

Response headers

content-type: text/plain; charset=utf-8

date: Sat, 03 Oct 2026 21:43:09 GMT

server: Kestrel

transfer-encoding: chunked

Responses

Teste 16: Atualizar quilometragem do Civic

Chamada	PUT /api/Veiculos/{id} id = 1
Entrada	<code>{"id":1,"modelo":"Civic","anoFabricacao":2022,"quilometragem":30500,"fabricanteId":1}</code>
Esperado	HTTP 200
Obtido	HTTP 200 — Veículo atualizado!
Resultado	Aprovado

PUT /api/Veiculos/{id}

Parameters

Cancel

Reset

Name	Description
id * required	
integer(\$int32)	1
(path)	

Request body

application/json

Edit Value | Schema

```
{  "id": 1,  "modelo": "Civic",  "anoFabricacao": 2022,  "quilometragem": 30500,  "fabricanteId": 1}
```

ExecuteClear

Responses

Request URL

http://localhost:5127/api/Veiculos/1

Server response

Code	Details
200	Response body Veículo atualizado! Download Response headers content-type: text/plain; charset=utf-8 date: Sat, 03 Oct 2026 21:43:09 GMT server: Kestrel transfer-encoding: chunked

Responses

Teste 17: Cadastrar cliente Carlos Almeida

Chamada	POST /api/Clientes
Entrada	<code>{"nome": "Carlos Almeida", "cpf": "111.222.333-44", "email": "carlos.almeida@email.com"}</code>
Esperado	HTTP 200
Obtido	HTTP 200 — <code>{"id": 1, "nome": "Carlos Almeida", "cpf": "111.222.333-44", "email": "carlos.almeida@email.com"}</code>
Resultado	Aprovado

POST /api/Clientes

Parameters

Cancel

Reset

No parameters

Request body

application/json

Edit Value | Schema

```
{
  "nome": "Carlos Almeida",
  "cpf": "111.222.333-44",
  "email": "carlos.almeida@email.com"
}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Clientes

Server response

Code	Details
200	Response body <pre>{ "id": 1, "nome": "Carlos Almeida", "cpf": "111.222.333-44", "email": "carlos.almeida@email.com" }</pre> Download Response headers content-type: application/json; charset=utf-8 date: Sat, 03 Oct 2026 21:43:10 GMT server: Kestrel transfer-encoding: chunked

Responses

Teste 18: Cadastrar cliente Ana Beatriz

Chamada	POST /api/Clientes
Entrada	<code>{"nome": "Ana Beatriz", "cpf": "555.666.777-88", "email": "ana.beatriz@email.com"}</code>
Esperado	HTTP 200
Obtido	HTTP 200 — <code>{"id": 2, "nome": "Ana Beatriz", "cpf": "555.666.777-88", "email": "ana.beatriz@email.com"}</code>
Resultado	Aprovado

POST /api/Clientes

Parameters

Cancel

Reset

No parameters

Request body

application/json

Edit Value | Schema

```
{  "nome": "Ana Beatriz",  "cpf": "555.666.777-88",  "email": "ana.beatriz@email.com"}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Clientes

Server response

Code	Details
200	Response body <pre>{ "id": 2, "nome": "Ana Beatriz", "cpf": "555.666.777-88", "email": "ana.beatriz@email.com"}</pre> Download Response headers content-type: application/json; charset=utf-8 date: Sat, 03 Oct 2026 21:43:11 GMT server: Kestrel transfer-encoding: chunked

Responses

Teste 19: Cadastrar cliente com CPF já existente

Chamada	POST /api/Clientes
Entrada	<code>{"nome": "Carlos Duplicado", "cpf": "111.222.333-44", "email": "outro@email.com"}</code>
Esperado	HTTP 409
Obtido	HTTP 409 — CPF já cadastrado.
Resultado	Aprovado

POST /api/Clientes

Parameters

Cancel

Reset

No parameters

Request body

application/json

Edit Value | Schema

```
{
  "nome": "Carlos Duplicado",
  "cpf": "111.222.333-44",
  "email": "outro@email.com"
}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Clientes

Server response

Code

Details

409

Error: Conflict

Undocumented

Response body

CPF já cadastrado.

Download

Response headers

content-type: text/plain; charset=utf-8
date: Sat, 03 Oct 2026 21:43:13 GMT
server: Kestrel
transfer-encoding: chunked

Responses

Teste 20: Cadastrar cliente com e-mail inválido

Chamada	POST /api/Clientes
Entrada	<code>{"nome": "Pedro Lima", "cpf": "999.888.777-66", "email": "pedro.email.com"}</code>
Esperado	HTTP 400
Obtido	HTTP 400 — <code>{"type": "https://tools.ietf.org/html/rfc9110#section-15.5.1", "title": "One or more validation errors occurred.", "status": 400, "errors": {"Email": ["The Email field is not a valid e-mail address."]}, "traceId": "00-b2cd9cd4174ea3a36b5eebaa9c91e83c-1feb96b34255f29b-00"}</code>
Resultado	Aprovado

POST /api/Clientes

Parameters

Cancel

Reset

No parameters

Request body

application/json

Edit Value | Schema

```
{  "nome": "Pedro Lima",  "cpf": "999.888.777-66",  "email": "pedro.email.com"}  
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Clientes

Server response

Code

Details

400
Undocumented

Error: Bad Request

Response body

```
{  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",  "title": "One or more validation errors occurred.",  "status": 400,  "errors": {    "Email": [      "The Email field is not a valid e-mail address."    ]  },  "traceId": "00-b2cd9cd4174ea3a36b5eebaa9c91e83c-1feb96b34255f29b-00"}  
```

Download

Response headers

```
content-type: application/problem+json; charset=utf-8date: Sat, 03 Oct 2026 21:43:14 GMTserver: Kestreltransfer-encoding: chunked
```

Responses

Teste 21: Listar todos os clientes

Chamada	GET /api/Clientes
Entrada	—
Esperado	HTTP 200
Obtido	HTTP 200 — [{"id":1,"nome":"Carlos Almeida","cpf":"111.222.333-44","email":"carlos.almeida@email.com"}, {"id":2,"nome":"Ana Beatriz","cpf":"555.666.777-88","email":"ana.beatriz@email.com"}]
Resultado	Aprovado

GET /api/Clientes

Parameters

No parameters

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Clientes

Server response

Code

Details

200

Response body

```
[
  {
    "id": 1,
    "nome": "Carlos Almeida",
    "cpf": "111.222.333-44",
    "email": "carlos.almeida@email.com"
  },
  {
    "id": 2,
    "nome": "Ana Beatriz",
    "cpf": "555.666.777-88",
    "email": "ana.beatriz@email.com"
  }
]
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Sat, 03 Oct 2026 21:43:15 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Teste 22: Buscar cliente por Id

Chamada	GET /api/Clientes/{id} id = 1
Entrada	—
Esperado	HTTP 200
Obtido	HTTP 200 — {"id":1,"nome":"Carlos Almeida","cpf":"111.222.333-44","email":"carlos.almeida@email.com"}
Resultado	Aprovado

GET /api/Clientes/{id}

Parameters

Cancel

Name	Description
id * required	
integer(\$int32)	1
(path)	

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Clientes/1

Server response

Code	Details
200	Response body <pre>{ "id": 1, "nome": "Carlos Almeida", "cpf": "111.222.333-44", "email": "carlos.almeida@email.com" }</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Sat, 03 Oct 2026 21:43:15 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Teste 23: Atualizar e-mail da cliente Ana

Chamada	PUT /api/Clientes/{id} id = 2
Entrada	<code>{"id":2,"nome":"Ana Beatriz","cpf":"555.666.777-88","email":"ana.beatriz.souza@email.com"}</code>
Esperado	HTTP 200
Obtido	HTTP 200 — Cliente atualizado!
Resultado	Aprovado

PUT /api/Clientes/{id}

Parameters

Cancel

Reset

Name	Description
id * required	
integer(\$int32)	2
(path)	

Request body

application/json

Edit Value | Schema

```
{
  "id": 2,
  "nome": "Ana Beatriz",
  "cpf": "555.666.777-88",
  "email": "ana.beatriz.souza@email.com"
}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Clientes/2

Server response

Code	Details
200	Response body Cliente atualizado! Download Response headers content-type: text/plain; charset=utf-8 date: Sat, 03 Oct 2026 21:43:16 GMT server: Kestrel transfer-encoding: chunked

Responses

Teste 24: Registrar aluguel: Carlos aluga o Civic

Chamada	POST /api/Alugueis
Entrada	<pre>{ "dataInicio": "2026-09-20T09:00:00", "dataFimPrevista": "2026-09-23T09:00:00", "quilometragemInicial": 30500, "valorDiaria": 120, "clienteId": 1, "veiculoId": 1 }</pre>
Esperado	HTTP 200
Obtido	HTTP 200 — <pre>{ "id": 1, "dataInicio": "2026-09-20T09:00:00", "dataFimPrevista": "2026-09-23T09:00:00", "dataDevolucao": null, "quilometragemInicial": 30500, "quilometragemFinal": null, "valorDiaria": 120, "valorTotal": null, "clienteId": 1, "veiculoId": 1 }</pre>
Resultado	Aprovado

POST /api/Alugueis

Parameters

Cancel

Reset

No parameters

Request body

application/json

Edit Value | Schema

```
{  "dataInicio": "2026-09-20T09:00:00",  "dataFimPrevista": "2026-09-23T09:00:00",  "quilometragemInicial": 30500,  "valorDiaria": 120,  "clienteId": 1,  "veiculoId": 1}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Alugueis

Server response

Code

Details

200

Response body

```
{  "id": 1,  "dataInicio": "2026-09-20T09:00:00",  "dataFimPrevista": "2026-09-23T09:00:00",  "dataDevolucao": null,  "quilometragemInicial": 30500,  "quilometragemFinal": null,  "valorDiaria": 120,  "valorTotal": null,  "clienteId": 1,  "veiculoId": 1}
```

Download

Response headers

content-type: application/json; charset=utf-8
date: Sat, 03 Oct 2026 21:43:16 GMT
server: Kestrel
transfer-encoding: chunked

Responses

Teste 25: Registrar aluguel: Ana aluga o Corolla

Chamada	POST /api/Alugueis
Entrada	<pre>{ "dataInicio": "2026-10-01T10:00:00", "dataFimPrevista": "2026-10-05T10:00:00", "quilometragemInicial": 12000, "valorDiaria": 150, "clienteId": 2, "veiculoId": 2 }</pre>
Esperado	HTTP 200
Obtido	HTTP 200 — <pre>{ "id": 2, "dataInicio": "2026-10-01T10:00:00", "dataFimPrevista": "2026-10-05T10:00:00", "dataDevolucao": null, "quilometragemInicial": 12000, "quilometragemFinal": null, "valorDiaria": 150, "valorTotal": null, "clienteId": 2, "veiculoId": 2 }</pre>
Resultado	Aprovado

POST /api/Alugueis

ParametersCancelReset

No parameters

Request bodyapplication/json

Edit ValueSchema

```
{  "dataInicio": "2026-10-01T10:00:00",  "dataFimPrevista": "2026-10-05T10:00:00",  "quilometragemInicial": 12000,  "valorDiaria": 150,  "clienteId": 2,  "veiculoId": 2}
```

ExecuteClear

Responses

Request URL

http://localhost:5127/api/Alugueis

Server response

Code	Details
200	Response body<pre>{ "id": 2, "dataInicio": "2026-10-01T10:00:00", "dataFimPrevista": "2026-10-05T10:00:00", "dataDevolucao": null, "quilometragemInicial": 12000, "quilometragemFinal": null, "valorDiaria": 150, "valorTotal": null, "clienteId": 2, "veiculoId": 2}</pre> Response headers<pre>content-type: application/json; charset=utf-8date: Sat, 03 Oct 2026 21:43:18 GMTserver: Kestreltransfer-encoding: chunked</pre>

Responses

Teste 26: Registrar aluguel com cliente inexistente

Chamada	POST /api/Alugueis
Entrada	<pre>{ "dataInicio": "2026-10-01T10:00:00", "dataFimPrevista": "2026-10-05T10:00:00", "quilometragemInicial": 0, "valorDiaria": 100, "clienteId": 999, "veiculoId": 1 }</pre>
Esperado	HTTP 400
Obtido	HTTP 400 — Cliente não existe.
Resultado	Aprovado

POST /api/Alugueis

Parameters

Cancel

Reset

No parameters

Request body

application/json

Edit Value

Schema

```
{  "dataInicio": "2026-10-01T10:00:00",  "dataFimPrevista": "2026-10-05T10:00:00",  "quilometragemInicial": 0,  "valorDiaria": 100,  "clienteId": 999,  "veiculoId": 1}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Alugueis

Server response

Code

Details

400

Undocumented

Error: Bad Request

Response body

Cliente não existe.

Download

Response headers

```
content-type: text/plain; charset=utf-8
date: Sat, 03 Oct 2026 21:43:19 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Teste 27: Listar todos os aluguéis

Chamada	GET /api/Alugueis
Entrada	—
Esperado	HTTP 200
Obtido	HTTP 200 — [{"id":1,"dataInicio":"2026-09-20T09:00:00","dataFimPrevista":"2026-09-23T09:00:00","dataDevolucao":null,"quilometragemInicial":30500,"quilometragemFinal":null,"valorDiaria":120,"valorTotal":null,"clienteId":1,"veiculoId":1},{ "id":2,"dataInicio":"2026-10-01T10:00:00","dataFimPrevista":"2026-10-05T10:00:00","dataDevolucao":null,"quilometragemInicial":12000,"quilometragemFinal":null,"valorDiaria":15 ...
Resultado	Aprovado

GET /api/Alugueis

Parameters

Cancel

No parameters

ExecuteClear

Responses

Request URL

http://localhost:5127/api/Alugueis

Server response

Code	Details
200	Response body <pre>[{ "id": 1, "dataInicio": "2026-09-20T09:00:00", "dataFimPrevista": "2026-09-23T09:00:00", "dataDevolucao": null, "quilometragemInicial": 30500, "quilometragemFinal": null, "valorDiaria": 120, "valorTotal": null, "clienteId": 1, "veiculoId": 1 }, { "id": 2, "dataInicio": "2026-10-01T10:00:00", "dataFimPrevista": "2026-10-05T10:00:00", "dataDevolucao": null, "quilometragemInicial": 12000, "quilometragemFinal": null, "valorDiaria": 150, "valorTotal": null, "clienteId": 2, "veiculoId": 2 }]</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Sat, 03 Oct 2026 21:43:20 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Teste 28: Buscar aluguel por Id

Chamada	GET /api/Alugueis/{id} id = 1
Entrada	—
Esperado	HTTP 200
Obtido	HTTP 200 — {"id":1,"dataInicio":"2026-09-20T09:00:00","dataFimPrevista":"2026-09-23T09:00:00","dataDevolucao":null,"quilometragemInicial":30500,"quilometragemFinal":null,"valorDiaria":120,"valorTotal":null,"clienteId":1,"veiculoId":1}
Resultado	Aprovado

GET /api/Alugueis/{id}

Parameters

Cancel

Name	Description
id ★ required	
integer(\$int32)	1
(path)	

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Alugueis/1

Server response

Code	Details
200	Response body <pre>{ "id": 1, "dataInicio": "2026-09-20T09:00:00", "dataFimPrevista": "2026-09-23T09:00:00", "dataDevolucao": null, "quilometragemInicial": 30500, "quilometragemFinal": null, "valorDiaria": 120, "valorTotal": null, "clienteId": 1, "veiculoId": 1 }</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Sat, 03 Oct 2026 21:43:21 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Teste 29: Registrar a devolução do Civic (data, km final e valor total)

Chamada	PUT /api/Alugueis/{id} id = 1
Entrada	<pre>{ "id": 1, "dataInicio": "2026-09-20T09:00:00", "dataFimPrevista": "2026-09-23T09:00:00", "dataDevolucao": "2026-09-23T08:30:00", "quilometragemInicial": 30500, "quilometragemFinal": 31120, "valorDiaria": 120, "valorTotal": 360, "clienteId": 1, "veiculoId": 1 }</pre>
Esperado	HTTP 200
Obtido	HTTP 200 — Aluguel atualizado!
Resultado	Aprovado

PUT

/api/Alugueis/{id}

⌵

Parameters

Cancel

Reset

Name	Description
id ★ required	
integer(\$int32)	1
(path)	

Request body

application/json

Edit Value

Schema

```
{  "id": 1,  "dataInicio": "2026-09-20T09:00:00",  "dataFimPrevista": "2026-09-23T09:00:00",  "dataDevolucao": "2026-09-23T08:30:00",  "quilometragemInicial": 30500,  "quilometragemFinal": 31120,  "valorDiaria": 120,  "valorTotal": 360,  "clienteId": 1,  "veiculoId": 1}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Alugueis/1

Server response

Code	Details
200	Response body Aluguel atualizado! Download Response headers content-type: text/plain; charset=utf-8 date: Sat, 03 Oct 2026 21:43:21 GMT server: Kestrel transfer-encoding: chunked

Responses

Teste 30: Registrar pagamento do aluguel do Carlos (PIX)

Chamada	POST /api/Pagamentos
Entrada	<code>{"metodoPagamento":"PIX","valorPago":360,"dataPagamento":"2026-09-23T09:00:00","aluguelId":1}</code>
Esperado	HTTP 200
Obtido	HTTP 200 — <code>{"id":1,"metodoPagamento":"PIX","valorPago":360,"dataPagamento":"2026-09-23T09:00:00","aluguelId":1}</code>
Resultado	Aprovado

POST /api/Pagamentos

Parameters

No parameters

Cancel

Reset

Request body

application/json

Edit Value | Schema

```
{
  "metodoPagamento": "PIX",
  "valorPago": 360,
  "dataPagamento": "2026-09-23T09:00:00",
  "aluguelId": 1
}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Pagamentos

Server response

Code

Details

200

Response body

```
{
  "id": 1,
  "metodoPagamento": "PIX",
  "valorPago": 360,
  "dataPagamento": "2026-09-23T09:00:00",
  "aluguelId": 1
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Sat, 03 Oct 2026 21:43:22 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Teste 31: Registrar pagamento de aluguel inexistente

Chamada	POST /api/Pagamentos
Entrada	<code>{"metodoPagamento":"PIX","valorPago":100,"dataPagamento":"2026-10-01T10:00:00","aluguelId":999}</code>
Esperado	HTTP 400
Obtido	HTTP 400 — Aluguel não existe.
Resultado	Aprovado

POST /api/Pagamentos

Parameters

Cancel

Reset

No parameters

Request body

application/json

Edit Value | Schema

```
{  "metodoPagamento": "PIX",  "valorPago": 100,  "dataPagamento": "2026-10-01T10:00:00",  "aluguelId": 999}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Pagamentos

Server response

Code	Details
400	Error: Bad Request

Undocumented

Response body

Aluguel não existe.

Download

Response headers

content-type: text/plain; charset=utf-8
date: Sat, 03 Oct 2026 21:43:23 GMT
server: Kestrel
transfer-encoding: chunked

Responses

Teste 32: Listar todos os pagamentos

Chamada	GET /api/Pagamentos
Entrada	—
Esperado	HTTP 200
Obtido	HTTP 200 — [{"id":1,"metodoPagamento":"PIX","valorPago":360,"dataPagamento":"2026-09-23T09:00:00","aluguelId":1}]
Resultado	Aprovado

GET /api/Pagamentos

Parameters

Cancel

No parameters

ExecuteClear

Responses

Request URL

http://localhost:5127/api/Pagamentos

Server response

CodeDetails

200

Response body

[{"id":1,"metodoPagamento":"PIX","valorPago":360,"dataPagamento":"2026-09-23T09:00:00","aluguelId":1}]

Response headers

content-type: application/json; charset=utf-8
date: Sat, 03 Oct 2026 21:43:24 GMT
server: Kestrel
transfer-encoding: chunked

Responses

RentalSystem — Relatório de Testes · página 40 de 59

Teste 33: Buscar pagamento por Id

Chamada	GET /api/Pagamentos/{id} id = 1
Entrada	—
Esperado	HTTP 200
Obtido	HTTP 200 — { "id":1, "metodoPagamento": "PIX", "valorPago": 360, "dataPagamento": "2026-09-23T09:00:00", "aluguelId": 1 }
Resultado	Aprovado

GET /api/Pagamentos/{id}

Parameters

Cancel

Name	Description
id ★ required	
integer(\$int32)	1
(path)	

ExecuteClear

Responses

Request URL

http://localhost:5127/api/Pagamentos/1

Server response

Code	Details
200	Response body <pre>{ "id": 1, "metodoPagamento": "PIX", "valorPago": 360, "dataPagamento": "2026-09-23T09:00:00", "aluguelId": 1 }</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Sat, 03 Oct 2026 21:43:24 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Teste 34: Atualizar método de pagamento para Cartão de Crédito

Chamada	PUT /api/Pagamentos/{id} id = 1
Entrada	<code>{"id":1,"metodoPagamento":"Cartão de Crédito","valorPago":360,"dataPagamento":"2026-09-23T09:00:00","aluguelId":1}</code>
Esperado	HTTP 200
Obtido	HTTP 200 — Pagamento atualizado!
Resultado	Aprovado

PUT /api/Pagamentos/{id}

Parameters

Cancel

Reset

Name	Description
id * required	
integer(\$int32)	1
(path)	

Request body

application/json

Edit Value | Schema

```
{  "id": 1,  "metodoPagamento": "Cartão de Crédito",  "valorPago": 360,  "dataPagamento": "2026-09-23T09:00:00",  "aluguelId": 1}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Pagamentos/1

Server response

Code	Details
200	Response body Pagamento atualizado!  Download Response headers content-type: text/plain; charset=utf-8 date: Sat, 03 Oct 2026 21:43:25 GMT server: Kestrel transfer-encoding: chunked

Responses

Teste 35: Veículos com fabricantes (INNER JOIN)

Chamada	GET /api/Filtros/veiculos-com-fabricantes
Entrada	—
Esperado	HTTP 200
Obtido	HTTP 200 — [{"id":1,"modelo":"Civic","anoFabricacao":2022,"fabricante":"Honda"}, {"id":2,"modelo":"Corolla","anoFabricacao":2023,"fabricante":"Toyota"}]
Resultado	Aprovado

GET /api/Filtros/veiculos-com-fabricantes

Parameters

No parameters

Execute Clear

Responses

Request URL

http://localhost:5127/api/Filtros/veiculos-com-fabricantes

Server response

Code

Details

200

Response body

[{"id":1,"modelo":"Civic","anoFabricacao":2022,"fabricante":"Honda"}, {"id":2,"modelo":"Corolla","anoFabricacao":2023,"fabricante":"Toyota"}]

Download

Response headers

content-type: application/json; charset=utf-8
date: Sat, 03 Oct 2026 21:43:26 GMT
server: Kestrel
transfer-encoding: chunked

Responses

Teste 36: Aluguéis detalhados com cliente e veículo (INNER JOIN)

Chamada	GET /api/Filtros/alugueis-detalhados
Entrada	—
Esperado	HTTP 200
Obtido	HTTP 200 — [{"idAluguel":1,"dataInicio":"2026-09-20T09:00:00","clienteNome":"Carlos Almeida","veiculoModelo":"Civic","valorTotal":360}, {"idAluguel":2,"dataInicio":"2026-10-01T10:00:00","clienteNome":"Ana Beatriz","veiculoModelo":"Corolla","valorTotal":null}]
Resultado	Aprovado

GET /api/Filtros/alugueis-detalhados

Parameters

Cancel

No parameters

ExecuteClear

Responses

Request URL

http://localhost:5127/api/Filtros/alugueis-detalhados

Server response

CodeDetails

200

Response body

```
[
  {
    "idAluguel": 1,
    "dataInicio": "2026-09-20T09:00:00",
    "clienteNome": "Carlos Almeida",
    "veiculoModelo": "Civic",
    "valorTotal": 360
  },
  {
    "idAluguel": 2,
    "dataInicio": "2026-10-01T10:00:00",
    "clienteNome": "Ana Beatriz",
    "veiculoModelo": "Corolla",
    "valorTotal": null
  }
]
```

Download

Response headers

content-type: application/json; charset=utf-8
date: Sat, 03 Oct 2026 21:43:26 GMT
server: Kestrel
transfer-encoding: chunked

Responses

RentalSystem — Relatório de Testes · página 44 de 59

Teste 37: Pagamentos por cliente (INNER JOIN, sintaxe LINQ)

Chamada	GET /api/Filtros/clientes-pagamentos-linq
Entrada	—
Esperado	HTTP 200
Obtido	HTTP 200 — [{"nomeCliente":"Carlos Almeida","cpf":"111.222.333-44","dataAluguel":"2026-09-20T09:00:00","valorPago":360,"metodo":"Cartão de Crédito"}]
Resultado	Aprovado

GET /api/Filtros/clientes-pagamentos-linq

Parameters Cancel

No parameters

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Filtros/clientes-pagamentos-linq

Server response

Code

Details

200

Response body

[
 {
 "nomeCliente": "Carlos Almeida",
 "cpf": "111.222.333-44",
 "dataAluguel": "2026-09-20T09:00:00",
 "valorPago": 360,
 "metodo": "Cartão de Crédito"
 }
]

 Download

Response headers

content-type: application/json; charset=utf-8
date: Sat, 03 Oct 2026 21:43:27 GMT
server: Kestrel
transfer-encoding: chunked

Responses

Teste 38: Aluguéis pendentes de devolução (INNER JOIN + WHERE)

Chamada	GET /api/Filtros/alugueis-pendentes
Entrada	—
Esperado	HTTP 200
Obtido	HTTP 200 — [{"id":2,"cliente":"Ana Beatriz","veiculo":"Corolla","dataFimPrevista":"2026-10-05T10:00:00"}]
Resultado	Aprovado

GET /api/Filtros/alugueis-pendentes

Parameters

Cancel

No parameters

ExecuteClear

Responses

Request URL

http://localhost:5127/api/Filtros/alugueis-pendentes

Server response

Code	Details
200	Response body <pre>[{"id": 2, "cliente": "Ana Beatriz", "veiculo": "Corolla", "dataFimPrevista": "2026-10-05T10:00:00"}]</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Sat, 03 Oct 2026 21:43:28 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Teste 39: Fabricantes e veículos, incluindo sem veículos (LEFT JOIN)

Chamada	GET /api/Filtros/fabricantes-veiculos-leftjoin
Entrada	—
Esperado	HTTP 200
Obtido	HTTP 200 — [{"fabricante":"Honda","veiculo":"Civic"}, {"fabricante":"Toyota","veiculo":"Corolla"}, {"fabricante":"Fiat Automóveis","veiculo":"Sem veículos"}]
Resultado	Aprovado

GET /api/Filtros/fabricantes-veiculos-leftjoin

Parameters

Cancel

No parameters

ExecuteClear

Responses

Request URL

http://localhost:5127/api/Filtros/fabricantes-veiculos-leftjoin

Server response

Code	Details
200	Response body <pre>[{ "fabricante": "Honda", "veiculo": "Civic" }, { "fabricante": "Toyota", "veiculo": "Corolla" }, { "fabricante": "Fiat Automóveis", "veiculo": "Sem veículos" }]</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Sat, 03 Oct 2026 21:43:28 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Teste 40: Cadastrar fabricante temporário

Chamada	POST /api/Fabricantes
Entrada	<code>{"nome": "Fabricante Temporário"}</code>
Esperado	HTTP 200
Obtido	HTTP 200 — <code>{"id": 4, "nome": "Fabricante Temporário"}</code>
Resultado	Aprovado

POST /api/Fabricantes

Parameters

Cancel

Reset

No parameters

Request body

application/json

Edit Value | Schema

```
{  
  "nome": "Fabricante Temporário"  
}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Fabricantes

Server response

Code	Details
200	Response body <pre>{ "id": 4, "nome": "Fabricante Temporário" }</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Sat, 03 Oct 2026 21:43:29 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Teste 41: Cadastrar veículo temporário

Chamada	POST /api/Veiculos
Entrada	<code>{"modelo": "Veículo Temporário", "anoFabricacao": 2020, "quilometragem": 1000, "fabricanteId": 4}</code>
Esperado	HTTP 200
Obtido	HTTP 200 — <code>{"id": 3, "modelo": "Veículo Temporário", "anoFabricacao": 2020, "quilometragem": 1000, "fabricanteId": 4}</code>
Resultado	Aprovado

POST /api/Veiculos

Parameters

Cancel

Reset

No parameters

Request body

application/json

Edit Value | Schema

```
{
  "modelo": "Veículo Temporário",
  "anoFabricacao": 2020,
  "quilometragem": 1000,
  "fabricanteId": 4
}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Veiculos

Server response

Code

Details

200

Response body

```
{
  "id": 3,
  "modelo": "Veículo Temporário",
  "anoFabricacao": 2020,
  "quilometragem": 1000,
  "fabricanteId": 4
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Sat, 03 Oct 2026 21:43:29 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Teste 42: Cadastrar cliente temporário

Chamada	POST /api/Clientes
Entrada	<code>{"nome":"Cliente Temporário","cpf":"000.000.001-91","email":"temporario@email.com"}</code>
Esperado	HTTP 200
Obtido	HTTP 200 — <code>{"id":3,"nome":"Cliente Temporário","cpf":"000.000.001-91","email":"temporario@email.com"}</code>
Resultado	Aprovado

POST /api/Clientes

Parameters

Cancel

Reset

No parameters

Request body

application/json

Edit Value

Schema

```
{
  "nome": "Cliente Temporário",
  "cpf": "000.000.001-91",
  "email": "temporario@email.com"
}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Clientes

Server response

Code

Details

200

Response body

```
{
  "id": 3,
  "nome": "Cliente Temporário",
  "cpf": "000.000.001-91",
  "email": "temporario@email.com"
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Sat, 03 Oct 2026 21:43:30 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Teste 43: Registrar aluguel temporário

Chamada	POST /api/Alugueis
Entrada	<pre>{ "dataInicio": "2026-10-02T10:00:00", "dataFimPrevista": "2026-10-03T10:00:00", "quilometragemInicial": 1000, "valorDiaria": 99.9, "clienteId": 3, "veiculoId": 3 }</pre>
Esperado	HTTP 200
Obtido	HTTP 200 — <pre>{ "id": 3, "dataInicio": "2026-10-02T10:00:00", "dataFimPrevista": "2026-10-03T10:00:00", "dataDevolucao": null, "quilometragemInicial": 1000, "quilometragemFinal": null, "valorDiaria": 99.9, "valorTotal": null, "clienteId": 3, "veiculoId": 3 }</pre>
Resultado	Aprovado

POST /api/Alugueis

Parameters

Cancel

Reset

No parameters

Request body

application/json

Edit Value | Schema

```
{  "dataInicio": "2026-10-02T10:00:00",  "dataFimPrevista": "2026-10-03T10:00:00",  "quilometragemInicial": 1000,  "valorDiaria": 99.9,  "clienteId": 3,  "veiculoId": 3}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Alugueis

Server response

Code	Details
200	Response body <pre>{ "id": 3, "dataInicio": "2026-10-02T10:00:00", "dataFimPrevista": "2026-10-03T10:00:00", "dataDevolucao": null, "quilometragemInicial": 1000, "quilometragemFinal": null, "valorDiaria": 99.9, "valorTotal": null, "clienteId": 3, "veiculoId": 3}</pre> Download Response headers content-type: application/json; charset=utf-8 date: Sat, 03 Oct 2026 21:43:30 GMT server: Kestrel transfer-encoding: chunked

Responses

Teste 44: Registrar pagamento temporário

Chamada	POST /api/Pagamentos
Entrada	<code>{"metodoPagamento":"Dinheiro","valorPago":99.9,"dataPagamento":"2026-10-02T10:00:00","aluguelId":3}</code>
Esperado	HTTP 200
Obtido	HTTP 200 — <code>{"id":2,"metodoPagamento":"Dinheiro","valorPago":99.9,"dataPagamento":"2026-10-02T10:00:00","aluguelId":3}</code>
Resultado	Aprovado

POST /api/Pagamentos

Parameters

Cancel

Reset

No parameters

Request body

application/json

Edit Value

Schema

```
{  "metodoPagamento": "Dinheiro",  "valorPago": 99.9,  "dataPagamento": "2026-10-02T10:00:00",  "aluguelId": 3}
```

Execute

Clear

Responses

Request URL

http://localhost:5127/api/Pagamentos

Server response

Code

Details

200

Response body

```
{  "id": 2,  "metodoPagamento": "Dinheiro",  "valorPago": 99.9,  "dataPagamento": "2026-10-02T10:00:00",  "aluguelId": 3}
```

Download

Response headers

```
content-type: application/json; charset=utf-8date: Sat, 03 Oct 2026 21:43:31 GMTserver: Kestreltransfer-encoding: chunked
```

Responses

Teste 45: Excluir pagamento

Chamada	DELETE /api/Pagamentos/{id} id = 2
Entrada	—
Esperado	HTTP 200
Obtido	HTTP 200 — Pagamento removido!
Resultado	Aprovado

DELETE /api/Pagamentos/{id}

Parameters

Cancel

Name	Description
id * required	
integer(\$int32)	2
(path)	

ExecuteClear

Responses

Request URL

http://localhost:5127/api/Pagamentos/2

Server response

Code	Details
200	Response body Pagamento removido! Download Response headers content-type: text/plain; charset=utf-8 date: Sat, 03 Oct 2026 21:43:31 GMT server: Kestrel transfer-encoding: chunked

Responses

Teste 46: Excluir aluguel

Chamada	DELETE /api/Alugueis/{id} id = 3
Entrada	—
Esperado	HTTP 200
Obtido	HTTP 200 — Aluguel removido!
Resultado	Aprovado

DELETE /api/Alugueis/{id}

Parameters

Cancel

Name	Description
id * required	
integer(\$int32)	3
(path)	

ExecuteClear

Responses

Request URL

http://localhost:5127/api/Alugueis/3

Server response

Code	Details
200	Response body Aluguel removido! Download Response headers content-type: text/plain; charset=utf-8 date: Sat, 03 Oct 2026 21:43:32 GMT server: Kestrel transfer-encoding: chunked

Responses

Teste 47: Excluir veículo

Chamada	DELETE /api/Veiculos/{id} id = 3
Entrada	—
Esperado	HTTP 200
Obtido	HTTP 200 — Veículo removido!
Resultado	Aprovado

DELETE /api/Veiculos/{id}

Parameters

Cancel

Name	Description
id * required	
integer(\$int32)	3
(path)	

ExecuteClear

Responses

Request URL

http://localhost:5127/api/Veiculos/3

Server response

Code	Details
200	Response body Veículo removido! Download Response headers content-type: text/plain; charset=utf-8 date: Sat, 03 Oct 2026 21:43:33 GMT server: Kestrel transfer-encoding: chunked

Responses

Teste 48: Excluir cliente

Chamada	DELETE /api/Clientes/{id} id = 3
Entrada	—
Esperado	HTTP 200
Obtido	HTTP 200 — Cliente removido!
Resultado	Aprovado

DELETE /api/Clientes/{id}

Parameters

Cancel

Name	Description
id * required	
integer(\$int32)	3
(path)	

ExecuteClear

Responses

Request URL

http://localhost:5127/api/Clientes/3

Server response

Code	Details
200	Response body Cliente removido! Download Response headers content-type: text/plain; charset=utf-8 date: Sat, 03 Oct 2026 21:43:33 GMT server: Kestrel transfer-encoding: chunked

Responses

Teste 49: Excluir fabricante

Chamada	DELETE /api/Fabricantes/{id} id = 4
Entrada	—
Esperado	HTTP 200
Obtido	HTTP 200 — {"mensagem": "Fabricante removido com sucesso!"}
Resultado	Aprovado

DELETE /api/Fabricantes/{id}

Parameters

Cancel

Name	Description
id * required	
integer(\$int32)	4
(path)	

ExecuteClear

Responses

Request URL

http://localhost:5127/api/Fabricantes/4

Server response

Code	Details
200	Response body <pre>{ "mensagem": "Fabricante removido com sucesso!" }</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Sat, 03 Oct 2026 21:43:34 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Teste 50: Excluir novamente o mesmo fabricante (já removido)

Chamada	DELETE /api/Fabricantes/{id} id = 4
Entrada	—
Esperado	HTTP 404
Obtido	HTTP 404 — Fabricante não encontrado.
Resultado	Aprovado

DELETE /api/Fabricantes/{id}

Parameters

Cancel

Name	Description
id * required	
integer(\$int32)	4
(path)	

ExecuteClear

Responses

Request URL

http://localhost:5127/api/Fabricantes/4

Server response

Code	Details
404	Error: Not Found

Undocumented

Response body

Fabricante não encontrado.

Download

Response headers

content-type: text/plain; charset=utf-8
date: Sat, 03 Oct 2026 21:43:35 GMT
server: Kestrel
transfer-encoding: chunked

Responses

4. Conclusão

Foram executados **50 testes** pelo Swagger, cobrindo os métodos POST, GET (lista e por Id), PUT e DELETE dos cinco controllers (Fabricantes, Veículos, Clientes, Aluguéis e Pagamentos) e os cinco filtros com INNER JOIN e LEFT JOIN. **50 de 50 testes foram aprovados.**

A API valida os dados de entrada (400), informa registros inexistentes (404), impede CPF duplicado (409) e mantém a integridade dos relacionamentos, rejeitando referências a fabricantes, clientes, veículos e aluguéis que não existem.